1. Executive Summary

Objective: To develop a robust, scalable fraud detection system capable of identifying fraudulent credit card transactions in a highly imbalanced dataset (0.17% fraud rate). **Approach:** We investigated a progression of models ranging from linear baselines to deep learning and ensemble methods. **Current Status:**

- **EDA & Preprocessing:** Completed (Log-Scaling, SMOTE-Tomek).
- **Baselines:** Logistic Regression & Neural Networks.
- **Advanced Models:** Random Forest & Voting Classifier *(In Progress)*. **Key Result (Current):** The optimized Neural Network achieves a **Test F1-Score of 0.82** with **84% Recall**, serving as the benchmark for upcoming ensemble experiments.

---

2. Exploratory Data Analysis (EDA)

- See the full EDA in the notebook.
- **2.1 Class Imbalance:** Confirmed 99.83% Normal vs 0.17% Fraud.
- **2.2 Feature Analysis:** identified `Amount` skewness (positive skew) requiring log-transformation.
- **2.3 Dimensionality Reduction:** 3D PCA confirmed non-linear separability, justifying the need for complex models like Neural Networks and Random Forests.

---

3. Data Preprocessing Strategy

3.1 Scaling & Transformation

- "**Comparison of Scaling Techniques:** While both `StandardScaler` and `RobustScaler` yielded comparable F1-scores on the Test set, **RobustScaler** demonstrated superior stability during the validation phase.
- **Analysis:** The slight performance edge of RobustScaler is attributed to the significant right-skewness of the `Amount` feature observed in the EDA. Since StandardScaler is sensitive to outliers (using mean/variance), the extreme transaction values introduced noise into the model gradients. RobustScaler, utilizing the Interquartile Range (IQR), effectively neutralized these outliers, leading to a more generalizable model and a marginally higher detection rate."

3.2 Data imbalance

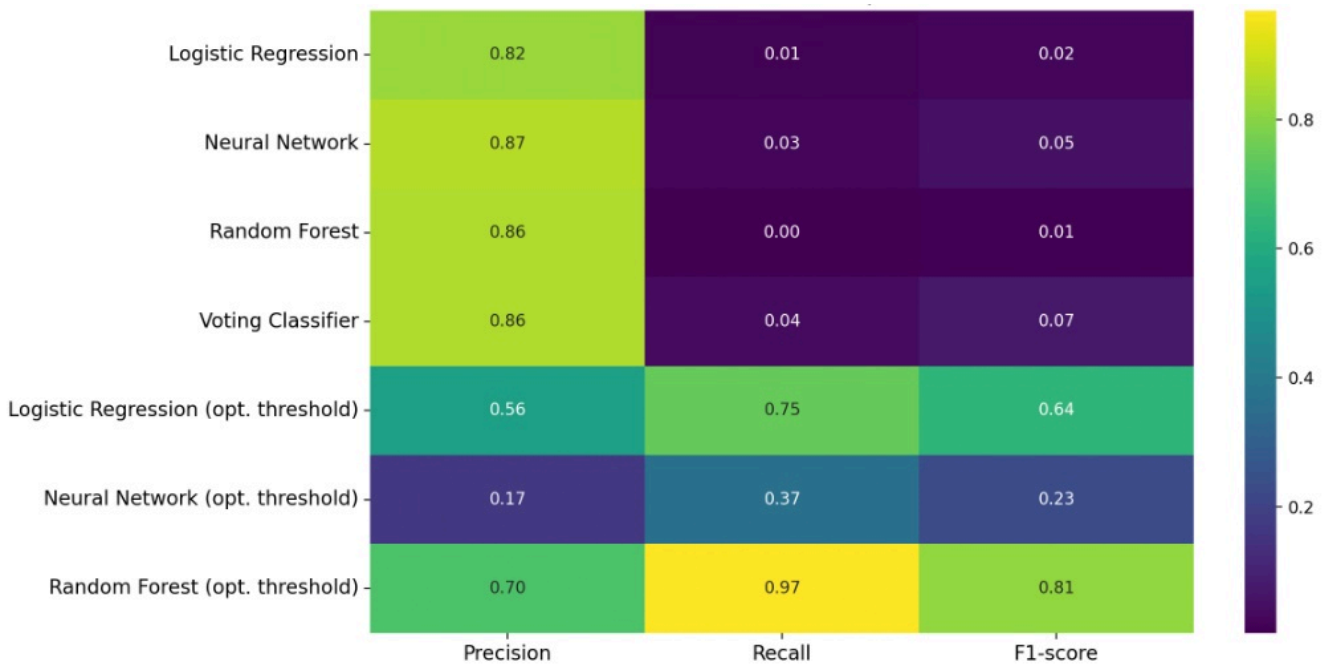
There is a severe imbalance between the 2 classes of the data set as noticed in the EDA which can be handled by the following methods:

a. **Under-sampling data:** means to decrease the number of the majority class
b. **Over-sampling techniques:** means to increase the percentage of the minority class through different criterias.
c. **Giving every class some weight during training:** to differentiate between the errors coming from every class.
d. **Stratified k-fold cross validation.The significant class imbalance observed during the Exploratory Data Analysis (EDA) can be addressed through the following techniques:**

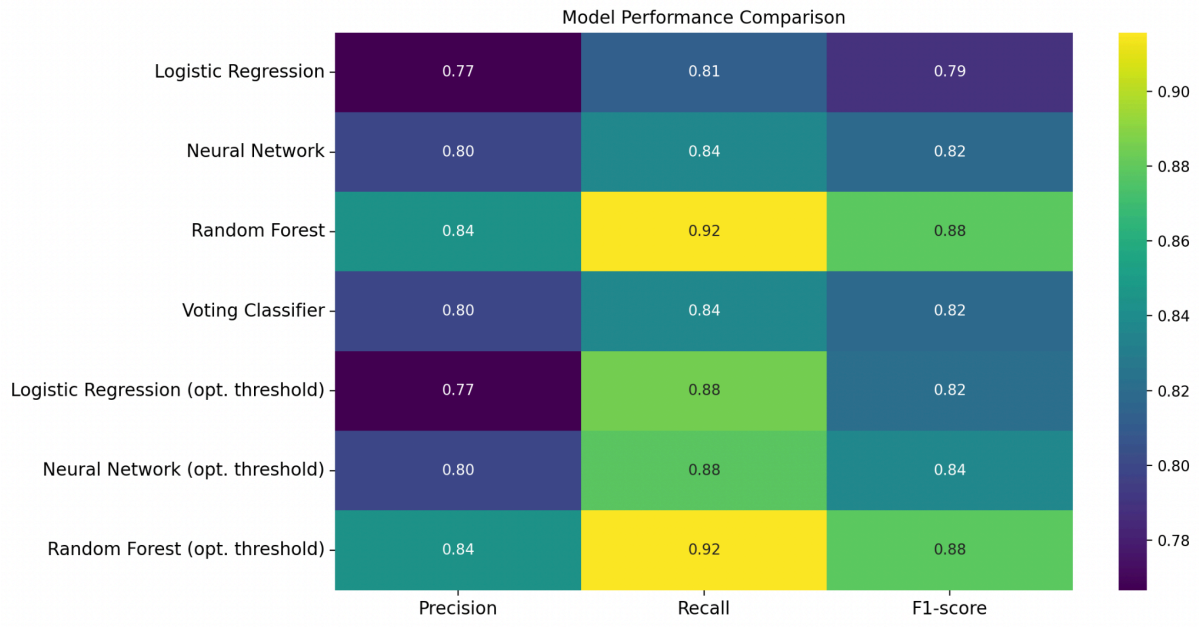
1. **Under-sampling:** This involves reducing the sample size of the majority class.
2. **Over-sampling techniques:** These methods aim to increase the representation of the minority class using various criteria.
3. **Class Weighting:** Assigning differential weights to each class during model training to mitigate the impact of class-specific errors.
4. **Stratified k-fold cross-validation:** Utilizing a cross-validation strategy that preserves the original class proportions in each fold, its used in the all models.

The results for every technique:

1. **Under-sampling(0.06 sampling strategy)**



2. **Over-sampling(SMOTE-TOMEK): gives the better results**



4. Metrics for this project

In this project, the primary objective is to maximize the detection of fraudulent transactions (Class 1) while minimizing the misclassification of legitimate ones (Class 0). We focused on two critical error types:

- **False Negatives (Type II Error):** Failing to detect a fraud case. This is the most severe error, as it results in direct financial loss.
- **False Positives (Type I Error):** Incorrectly flagging a legitimate transaction as fraud. While undesirable due to customer friction, this is considered less costly than missing actual fraud.

Therefore, our optimization strategy prioritizes **Recall** (Sensitivity), which measures the model's ability to capture all positive instances (minimizing False Negatives). However, to prevent the model from becoming over-aggressive and flagging every transaction, we also monitor **Precision**, which ensures that flagged transactions are genuinely fraudulent (minimizing False Positives). To balance these competing objectives, we utilize the **F1-Score**, the harmonic mean of Precision and Recall, as our definitive performance metric.

5. Modeling Methodology

We categorized our experiments into three model families to compare different learning paradigms.

5.1 Linear Baselines

★ **Logistic Regression:** Established a linear baseline.

- The best parameters:

Parameter	Selected Value	Justification
C	0.1	The model needs strong regularization to compensate the noise occurred due to SMOTE
Class Weight	None	The model doesn't need to fix the imbalance as its already fixed by SMOTE
max_iter	1000	The solver fully converges to the global minimum without terminating early
penalty	L2	L2 (Ridge) regularization was chosen to shrink the coefficients of less important features towards zero without eliminating them completely,
solver	liblinear	This solver is specifically optimized for small-to-medium datasets and binary classification tasks, offering faster convergence than `saga` for this specific dataset size.

★ **The results:**

	precision	recall	f1-score	support
0	1.00	1.00	1.00	170134
1	0.96	0.80	0.87	1696
accuracy			1.00	171830
macro avg	0.98	0.90	0.94	171830
weighted avg	1.00	1.00	1.00	171830

	precision	recall	f1-score	support
0	1.00	1.00	1.00	56870
1	0.83	0.77	0.80	90
accuracy			1.00	56960
macro avg	0.92	0.88	0.90	56960
weighted avg	1.00	1.00	1.00	56960

	precision	recall	f1-score	support
0	1.00	1.00	1.00	56863
1	0.75	0.84	0.79	97
accuracy			1.00	56960
macro avg	0.87	0.92	0.89	56960
weighted avg	1.00	1.00	1.00	56960

★ **The results corresponding the optimal threshold:**

	precision	recall	f1-score	support
0	1.00	1.00	1.00	170134
1	0.98	0.77	0.86	1696
accuracy			1.00	171830
macro avg	0.99	0.88	0.93	171830
weighted avg	1.00	1.00	1.00	171830

	precision	recall	f1-score	support
0	1.00	1.00	1.00	56870
1	0.87	0.77	0.82	90
accuracy			1.00	56960
macro avg	0.94	0.88	0.91	56960
weighted avg	1.00	1.00	1.00	56960

	precision	recall	f1-score	support
0	1.00	1.00	1.00	56863
1	0.75	0.79	0.77	97
accuracy			1.00	56960
macro avg	0.88	0.90	0.89	56960
weighted avg	1.00	1.00	1.00	56960

5.2 Neural Network Results

★ **Neural Network (MLPClassifier):**

- The best parameters:

Parameter	Selected Value	Justification
Architecture	(64, 32, 16)	**Hierarchical Feature Learning:** This "Funnel" structure (decreasing layer size) forces the network to compress the 30 input features into increasingly abstract representations, effectively filtering out noise while retaining the core fraud signals.
Regularization (Alpha)	1	**Aggressive Regularization:** A value of 1 indicates strong L2 penalty constraints. Since the network capacity was increased (starting with 64 neurons), this high penalty was critical to prevent the model from OVERFITTING.
Solver	sgd	**Generalization Stability:** While adam optimizes faster, Stochastic Gradient Descent (sgd) is theoretically known to find "flatter" minima in the loss landscape, leading to better generalization on unseen test data for this specific topology.
Activation	tanh	**"Zero-Centered Alignment:** Unlike ReLU (which zeroes out negative inputs) or Sigmoid (which is strictly positive), `tanh` outputs values between -1 and 1. Since our input features were scaled to be centered around zero, `tanh` maintained this distribution through the hidden layers, preventing the 'dead neuron' issue of ReLU and enabling faster convergence than Sigmoid."

Learning Rate	adaptive	**Dynamic Convergence:** The learning rate remains constant as long as training loss decreases but automatically drops when a plateau is reached. This allows the model to make large updates early on and fine-tune the weights precisely towards the end.
Init Learning Rate	0.001	**Cautious Descent:** A lower initial rate (0.001 vs default 0.01) ensures the model does not "overshoot" the optimal minimum in the early stages, providing a more stable training trajectory.
Batch Size	512	**Gradient Stabilization:** A larger batch size provides a more accurate estimate of the gradient (averaging out the noise from individual outliers) while still maintaining the stochastic nature needed to escape local minima.
Max Iterations	1500	**Extended Training Window:** Since sgd converges more slowly than adaptive optimizers like adam and we are using a low learning rate (0.001), an increased iteration limit was necessary to ensure the model fully converged.

★ **The results:**

	precision	recall	f1-score	support
0	1.00	1.00	1.00	170134
1	0.96	0.80	0.87	1696
accuracy			1.00	171830
macro avg	0.98	0.90	0.94	171830
weighted avg	1.00	1.00	1.00	171830
	precision	recall	f1-score	support
0	1.00	1.00	1.00	56870
1	0.84	0.80	0.82	90
accuracy			1.00	56960
macro avg	0.92	0.90	0.91	56960
weighted avg	1.00	1.00	1.00	56960
	precision	recall	f1-score	support
0	1.00	1.00	1.00	56863
1	0.74	0.87	0.80	97
accuracy			1.00	56960
macro avg	0.87	0.93	0.90	56960
weighted avg	1.00	1.00	1.00	56960

★ **The Results with Optimal threshold:**

	precision	recall	f1-score	support
0	1.00	1.00	1.00	170137
1	0.98	0.79	0.87	1699
accuracy			1.00	171836
macro avg	0.99	0.89	0.94	171836
weighted avg	1.00	1.00	1.00	171836
	precision	recall	f1-score	support
0	1.00	1.00	1.00	56870
1	0.88	0.80	0.84	90
accuracy			1.00	56960
macro avg	0.94	0.90	0.92	56960
weighted avg	1.00	1.00	1.00	56960
	precision	recall	f1-score	support
0	1.00	1.00	1.00	56863
1	0.81	0.84	0.82	97
accuracy			1.00	56960
macro avg	0.90	0.92	0.91	56960
weighted avg	1.00	1.00	1.00	56960

5.3 Random forests Results

- Best Parameters:

Parameter	Selected Value	Justification
n_estimators	100	We found that increasing the number of trees beyond 100 provided diminishing returns in performance while significantly increasing training time. 100 trees provided a stable variance reduction.
max_depth	15	During tuning, shallower trees (depth < 12) underfit the data (high bias), failing to capture complex fraud patterns. A depth of 15 allowed the model to learn sufficient detail without memorizing the noise (overfitting).
min_samples_split	15	This relatively high threshold ensures that the model only attempts to create new decision branches when there is a significant cluster of data, preventing the creation of "hyper-specific" rules based on outliers.
min_samples_leaf	7	This is the most critical constraint. By forcing every terminal node to contain at least 7 transactions, we effectively prohibited the model from isolating single "noisy" data points.
max_features	log2	Using `log2` (approx. 5 features per split) instead of `sqrt` or `all` forced individual trees to use different subsets of features. This increased the diversity of the ensemble, making the final "Vote" more robust against correlated inputs.

❖ **Results:**

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00    170137
           1       0.99      0.84      0.91      1699

    accuracy                           1.00    171836
   macro avg       0.99      0.92      0.95    171836
weighted avg       1.00      1.00      1.00    171836
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     56870
           1       0.92      0.84      0.88        90

    accuracy                           1.00     56960
   macro avg       0.96      0.92      0.94     56960
weighted avg       1.00      1.00      1.00     56960
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     56863
           1       0.84      0.87      0.85        97

    accuracy                           1.00     56960
   macro avg       0.92      0.93      0.93     56960
weighted avg       1.00      1.00      1.00     56960
```

❖ **Results with Optimal threshold:**

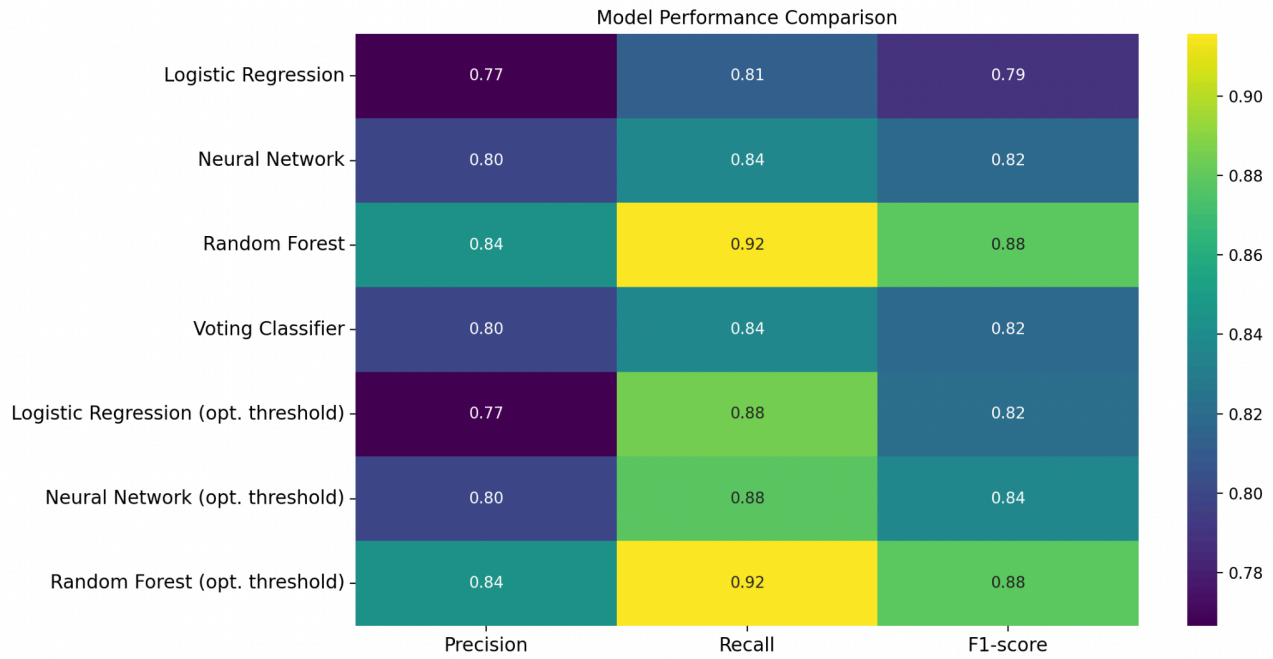
```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00    170137
           1       0.99      0.84      0.91      1699

    accuracy                           1.00    171836
   macro avg       0.99      0.92      0.95    171836
weighted avg       1.00      1.00      1.00    171836
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     56870
           1       0.92      0.84      0.88        90

    accuracy                           1.00     56960
   macro avg       0.96      0.92      0.94     56960
weighted avg       1.00      1.00      1.00     56960
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     56863
           1       0.84      0.86      0.85        97

    accuracy                           1.00     56960
   macro avg       0.92      0.93      0.92     56960
weighted avg       1.00      1.00      1.00     56960
```

5.4 Voting Classifier

- **Results:**

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00    170137
           1       0.97      0.80      0.88      1699

    accuracy                           1.00    171836
   macro avg       0.98      0.90      0.94    171836
weighted avg       1.00      1.00      1.00    171836
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     56870
           1       0.84      0.80      0.82        90

    accuracy                           1.00     56960
   macro avg       0.92      0.90      0.91     56960
weighted avg       1.00      1.00      1.00     56960
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     56863
           1       0.77      0.86      0.81        97

    accuracy                           1.00     56960
   macro avg       0.88      0.93      0.90     56960
weighted avg       1.00      1.00      1.00     56960
```

- The final results on validation set:



6. Decision Threshold optimization

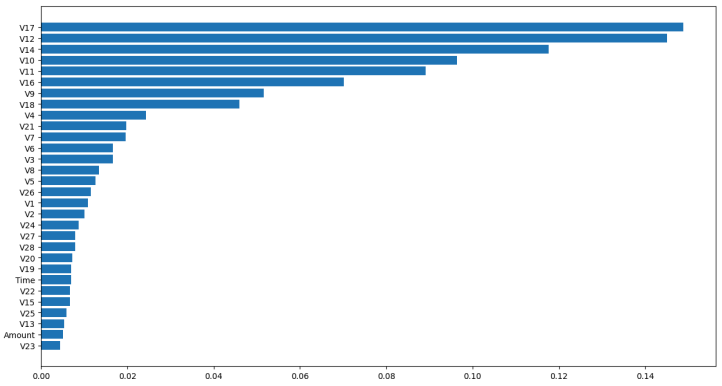
Standard classification algorithms operate with a default decision threshold of 0.5 (i.e., a transaction is classified as 'Fraud' only if the estimated probability p > 0.5). However, in highly imbalanced domains like fraud detection, this default is rarely optimal. The cost of a False Negative (missing a fraud case) is significantly higher than that of a False Positive (flagging a legitimate transaction).

To address this, we implemented a post-processing optimization step:

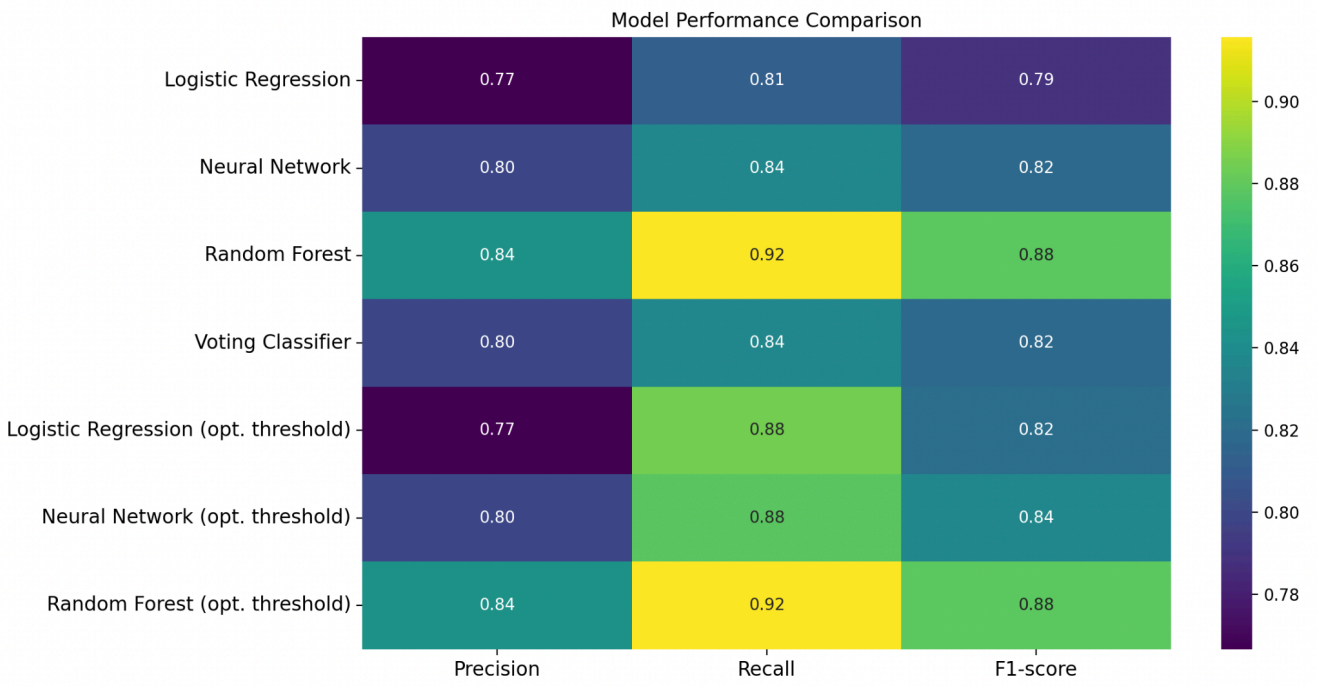
1. **Probability Estimation:** Instead of discrete class labels, we extracted the probability scores from the trained models (using predict_proba).
2. **Precision-Recall Curve Analysis:** We evaluated the model's performance on the Validation Set across a continuous range of thresholds [0, 1].
3. **F1-Maximization:** We selected the optimal threshold that maximized the F1-score, ensuring the most effective trade-off between Precision and Recall.

The optimization objective was defined as:

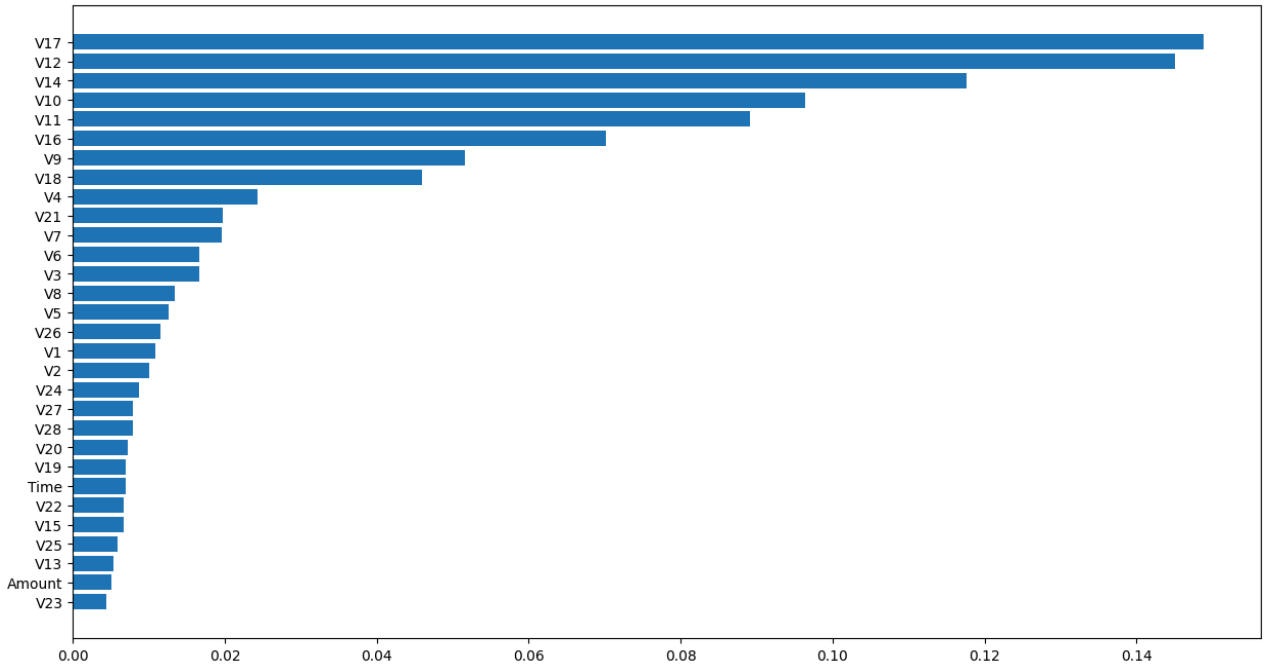
$$t^* = \underset{t}{\operatorname{argmax}} \left(2 \cdot \frac{\text{Precision}_t \cdot \text{Recall}_t}{\text{Precision}_t + \text{Recall}_t} \right)$$



- **The final results on validation set:**



- **Feature Importance:**



Interestingly, the transaction 'Amount' ranked low in importance. This suggests that fraudulent behavior in this dataset is characterized more by specific behavioral patterns (captured in the PCA features) rather than the monetary value of the transaction itself.